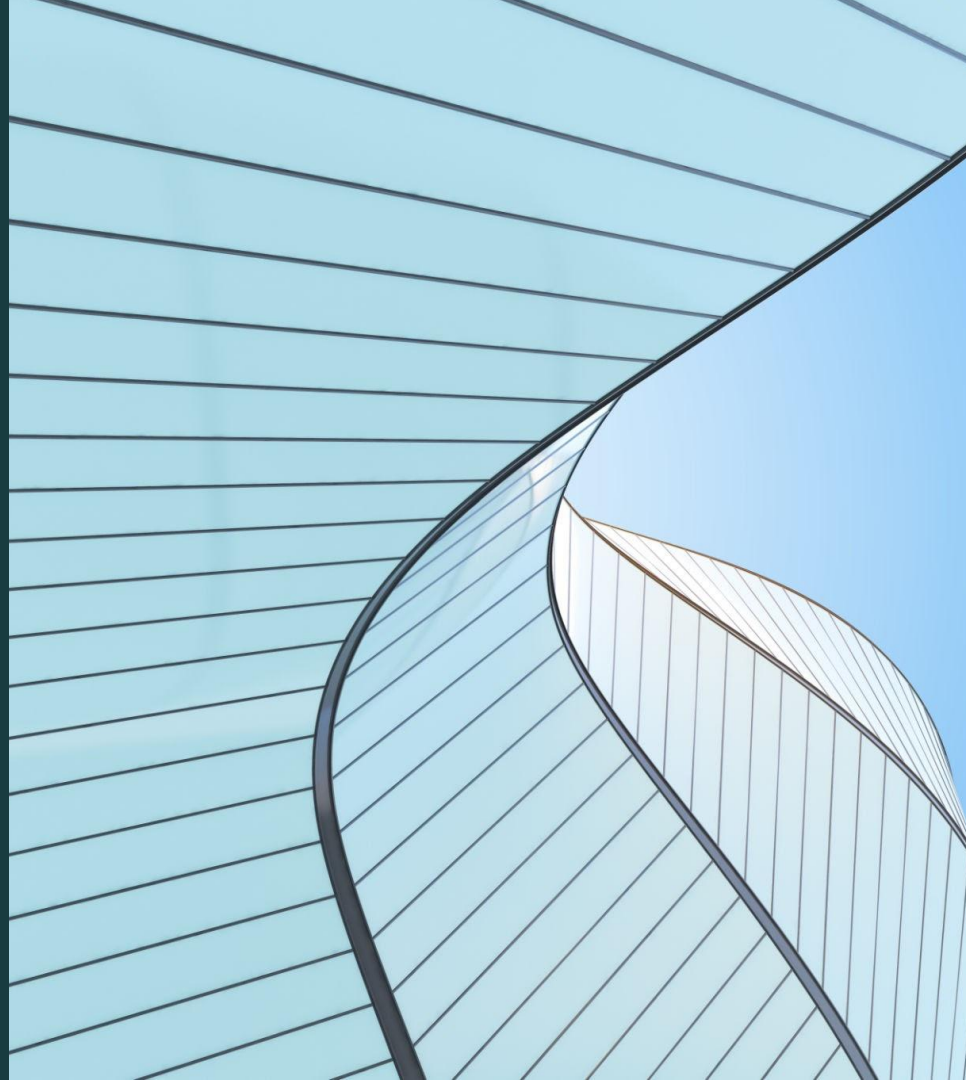
Austin Cregeur

AI Document Assistant

A Retrieval-Augmented Generation System for Querying Local Documents



PROBLEM STATEMENT

We store valuable information across multiple documents and folders. Manually searching through documents is too slow, and using basic keyword searches are insufficient. While cloud solutions exist, they present privacy issues for sensitive documents.

PROJECT OBJECTIVE

A RAG-based document assistant that:

1. Runs locally, eliminate privacy concerns
2. Supports PDF and TXT files
3. Implements Natural Language Retrieval, instead of exact keyword matching
4. Provides answers with references to source documents
5. Utilizes a simple interface

RAG Explained

Retrieval-Augmented Generation (RAG)

- Combines search + AI generation
- Retrieval = finding relevant document chunks
- Generation = LLM response from retrieved content

Why?

LLMs have no knowledge of our local files, the model needs context to get an accurate response.

How?

Embeddings & Vector Search → FAISS → LLM

Embeddings & Vector Search, FAISS

- Documents are broken into chunks and converted into embeddings
- Embeddings capture the meaning of the document chunks
- User queries are also converted into embeddings
- If a document is relevant to the query, their embeddings will be similar

Facebook AI Similarity Search (FAISS)

- FAISS is a vector similarity search library
- It finds relevant document chunks for our query

HIGH LEVEL PROJECT OVERVIEW

The system consists of 2 phases: ingestion and querying.

Ingestion:

- Documents are read from a local folder
- Content is split into manageable chunks
- Each chunk is converted into vector embeddings
- Stored in a FAISS index

Querying:

- Query is converted into an embedding
- Relevant document chunks are retrieved and reranked
- Context is passed into a prompt
- A local LLM generates a response with sources

TECH STACK

1. Backend - Python, FastAPI
2. Document Processing - pypdf Library
3. Vector Database - FAISS
4. LLM - qwen2.5:3b with Ollama
5. Embedding - SentenceTransformers with BAAI/bge-small-en-v1.5
6. Reranking - CrossEncoder with ms-marco-MiniLM-L-6-v2

Ingestion Pipeline: In-depth

1. POST request send to API endpoint with local folder path
2. Document ingestor walks through the folder, extracts text
3. Text is normalized and chunked (5000 chr chunk size, 100 overlap)
4. Chunks are passed to embedding model (BAAI/bge-small-en-v1.5)
5. Embeddings are stored into vector store, FAISS index created

Query Pipeline: In-depth

1. POST request sent to API endpoint with query
2. Check for any changed files since ingestion. If there are changes, rerun ingestion pipeline and continue
3. Query is embedded using same ingestion embedding model
4. Embedding sent to FAISS and retrieves similar chunk vectors
5. 2nd retrieval done using lexical search (useful for specific phrases, names, etc)
6. Merge results from both retrievals
7. Rerank the results using a cross-encoder (ms-marco-MiniLM-l-6-v2)
8. Top k chunks are selected from reranking, passed to LLM prompt
9. LLM response (with sources) is rendered to web page

Results

Based on a set of 10 benchmark queries from sample dataset:

Chunk Size	50	500	5000	50000
Time to Build Vector Database (s)	713.8	24.5	21.4	21.5
Query Success Rate	30%	80%	100%	100%
Correct Source is Top 1	60%	80%	100%	100%
Correct Source in Top 5	80%	100%	100%	100%

Notes on Results/Concluding Remarks

- Model struggles when:
  - Documents aren't verbose enough
  - Query phrasing/wording not similar enough to source document
  - Relevant information spread across multiple documents
- Chunk Size
  - Based on results, we should have one giant chunk, right?
  - In reality, the ideal chunk size depends on the dataset and the questions being asked
  - Very likely my dataset may have been too small and my questions too broad to truly evaluate the ideal chunk size for performance
- Future Improvements
  - Testing on a variety of datasets to better measure performance
  - Testing across varying chunk overlap size